Trabalho Prático 3: Sistema de Mensageria Segura Multi-Cliente

Disciplina: Segurança da Informação
Professor: Michel Sales

Objetivo

Implementar uma aplicação de **mensageria segura multi-cliente** com criptografia ponta-a-ponta (E2E). O servidor atua somente como relay, sem capacidade de decifrar mensagens. Cada cliente realiza um handshake autenticado com o servidor para registrar sua chave pública efêmera e receber as chaves públicas dos demais clientes. O sistema deve garantir **confidencialidade, integridade, autenticidade e sigilo perfeito**.

Requisitos de Segurança

1. **Confidencialidade**
   - As mensagens devem ser ilegíveis para terceiros.
   - **Mecanismo:** AES-128-GCM (AEAD).
2. **Integridade**
   - Alterações não autorizadas devem ser detectadas.
   - **Mecanismo:** Tag de autenticação do AES-GCM.
3. **Autenticidade**
   - O servidor deve ser autenticado pelos clientes. Recomendado: RSA-PSS.
   - **Mecanismo:** Certificado RSA autoassinado, usado para assinar a chave efêmera ECDHE.
4. **Sigilo Perfeito (Forward Secrecy)**
   - Mesmo que a chave RSA seja comprometida no futuro, as sessões passadas permanecem seguras.
   - **Mecanismo:** ECDHE (Elliptic Curve Diffie-Hellman Ephemeral).

1. Elementos do Protocolo

Identificadores

- **client_id (16 bytes)**: UUID v4 binário, identifica unicamente o cliente.
- **sender_id / recipient_id (16 bytes)**: IDs incluídos no frame E2E, usados no AAD do AES-GCM.

Chaves Criptográficas

- **sk_client (cliente)**: chave privada X25519 (32B), gerada localmente.
- **pk_client (cliente)**: chave pública X25519 (32B), enviada ao servidor.
- **sk_server (servidor)**: chave privada RSA (≥2048 bits), usada para assinar o handshake.
- **pk_server (servidor)**: distribuída no certificado `server.crt` .

Segredo Compartilhado E2E

- **Z_AB = ECDH(sk_A, pk_B)**
- Usado como IKM no HKDF. Nunca transmitido.

Chaves Simétricas E2E

Derivadas via HKDF-SHA256:

- `PRK = HKDF-Extract(salt = salt_srv, IKM = Z_AB)`
- `Key_A2B = HKDF-Expand(PRK, info="A2B", L=16)`
- `Key_B2A = HKDF-Expand(PRK, info="B2A", L=16)`

Salt

- **salt_srv (16 bytes)**: gerado pelo servidor no handshake.

Contadores e Nonces

- **seq_no (8 bytes)**: contador monotônico por direção.
- **nonce (12 bytes)**: `IV_base (4B) || seq_no (8B)`

Requisitos de Geração de Chaves

- **Curva ECDHE e formato de chaves:**
  - Use X25519 para ECDHE (32 bytes raw public key). Clientes geram pares efêmeros e publicam a chave pública (raw 32 bytes) ao servidor como parte do registro/handshake.
  - Especificar formato: chaves públicas X25519 enviadas como 32 bytes raw (não PEM/DER) para simplicidade e interoperabilidade.
- **Assinatura RSA (autenticação do servidor):**
  - O servidor possui um par RSA (recomendado 2048+ bits) e um certificado autoassinado `server.crt` que é *pinado* nos clientes (ou disponibilizado fora do canal inseguro).
  - Esquema de assinatura: RSA-PSS com SHA-256 (recomenda-se) sobre o hash SHA-256 do material a assinar.
  - Definição concreta: calcular H = SHA256(pk_server || pk_client || client_id || salt) (onde `pk_server` e `pk_client` são raw bytes, `client_id` é 16B binário e `salt` é 16B). O servidor assina H com RSA-PSS(SHA256) e envia a assinatura junto com `server.crt`.
- **HKDF (parâmetros):**
  - Use HKDF com HMAC-SHA256. Salt: 16 bytes aleatórios gerados pelo servidor quando necessário. Z (o output raw do ECDH X25519) é o IKM.
  - Aplicar: PRK = HKDF-Extract(salt, Z). Em seguida, gerar chaves com HKDF-Expand(PRK, info=label, L=16). Para comunicações E2E entre pares A e B, use labels ASCII `"A2B"` e `"B2A"` (consistente com a seção "Chaves Simétricas E2E").

Observação: neste trabalho a arquitetura passa a ser E2E: o servidor NÃO deve decifrar o conteúdo das mensagens (ver seção de alterações abaixo). O servidor atua como relay/rendezvous e não armazena chaves simétricas que permitam decifrar mensagens cliente→cliente.

Definições: sk / pk (chaves privadas e públicas)

- sk = secret key (chave privada): a chave secreta local de um participante. Nunca deve ser enviada pela rede, nunca deve ser logada e deve ser mantida apenas no dispositivo do dono da chave.
- pk = public key (chave pública): a chave pública correspondente que pode ser enviada ao peer e usada para calcular o segredo compartilhado.

Em fórmulas e exemplos use nomes explícitos para evitar ambiguidade: por exemplo `sk_client`, `pk_client`, `sk_server`, `pk_server`, ou `sk_A`, `pk_A`, `sk_B`, `pk_B`.

No cálculo ECDH, cada lado usa sua chave privada (sk) e a chave pública recebida do peer (pk). Para manter consistência com o restante do documento, escreva explicitamente por par:

Z_AB = ECDH(sk_A, pk_B)

Exemplo prático com X25519 (ilustrativo, Python/cryptography):

1. Geração de par efêmero (no cliente):
    - sk_client = X25519PrivateKey.generate() # chave privada — NÃO enviar
    - pk_client = sk_client.public_key().public_bytes(encoding=Encoding.Raw, format=PublicFormat.Raw) # 32 bytes — enviar ao peer
2. Ao receber pk_peer (32 bytes raw):
    - pk_peer_obj = X25519PublicKey.from_public_bytes(pk_peer_bytes)
    - Z = sk_client.exchange(pk_peer_obj) # shared secret bytes (IKM para HKDF)

Estrutura da Mensagem

```
[nonce (12B)] + [sender_id (16B)] + [recipient_id (16B)] + [seq_no (8B)] + [ciphertext+
```

- **AAD (Associated Data):** `sender_id | recipient_id | seq_no`
- **Proteção:** AES-128-GCM garante confidencialidade, integridade e autenticidade.
- **Nonce:** deve ser único por mensagem e por direção.
- **Seq_no:** contador monotônico para evitar ataques de replay.

Notas e especificações precisas:

- `sender_id` e `recipient_id` — 16 bytes binários: use UUID v4 codificado em 16 bytes (network byte order). Para debugging, uma representação textual UUID pode ser usada fora do protocolo.
- Nonce (12 bytes): construir como `IV_base (4B) || seq_no (8B)` onde IV_base é gerado aleatoriamente por sessão (4 bytes) e seq_no é o contador 64-bit big-endian. Alternativamente, usar `direction (4B) || seq_no (8B)` com direction = 0x00000000 para c2s e 0x00000001 para s2c. Especificar big-endian para seq_no.
- Framing: em TCP, cada frame deve ser prefixado por um comprimento de 4 bytes (big-endian) indicando o tamanho do frame que se segue. Isso evita problemas de delimitação de mensagens em streams.
- Ciphertext+tag: tag AES-GCM de 16 bytes deve ser concatenada ao final do ciphertext.

Estrutura de Dados no Servidor (Exemplo em Python)

```
sessions = {
   client_id: {
      "writer": StreamWriter,
      "public_key": pk_C (32B),
      "seq_recv": último seq_no recebido do cliente,
      "seq_send": próximo seq_no para mensagens de controle enviadas pelo servidor,
      "salt": salt_srv (16B)
   },
   ...
}
```

Função do `seq_send`

O servidor envia mensagens administrativas (não E2E), como:

- Confirmação de registro
- Avisos de desconexão
- Erros de framing
- Atualizações da lista de clientes

Essas mensagens precisam de ordenação e proteção contra replay. Por isso, o servidor mantém um contador separado: `seq_send` . Ele não interfere no E2E entre clientes.

Estrutura de Dados Mantida pelo Cliente

Cliente A:

```
peers = {
   client_id_B: {
      "public_key": pk_C_B (32B),
      "Z_AB": segredo ECDH com B (opcional armazenar),
      "key_send": Key_A2B (AES-128-GCM),
      "key_recv": Key_B2A (AES-128-GCM),
      "seq_send": próximo seq_no para enviar mensagens a B,
      "seq_recv": último seq_no recebido de B,
      "IV_base_send": IV_base para direção A→B,
      "IV_base_recv": IV_base para direção B→A
   },
   ...
}
```

Explicação dos campos:

- **public_key**: chave pública X25519 do peer.
- **Z_AB**: segredo ECDH (descartável após derivar chaves).
- **key_send / key_recv**: chaves simétricas derivadas via HKDF. `key_send` para enviar para B e `key_recv` para receber de B.
- **seq_send**: contador monotônico para mensagens enviadas.
- **seq_recv**: último seq_no aceito (anti-replay).
- **IV_base_send / IV_base_recv**: IV base aleatório por direção.

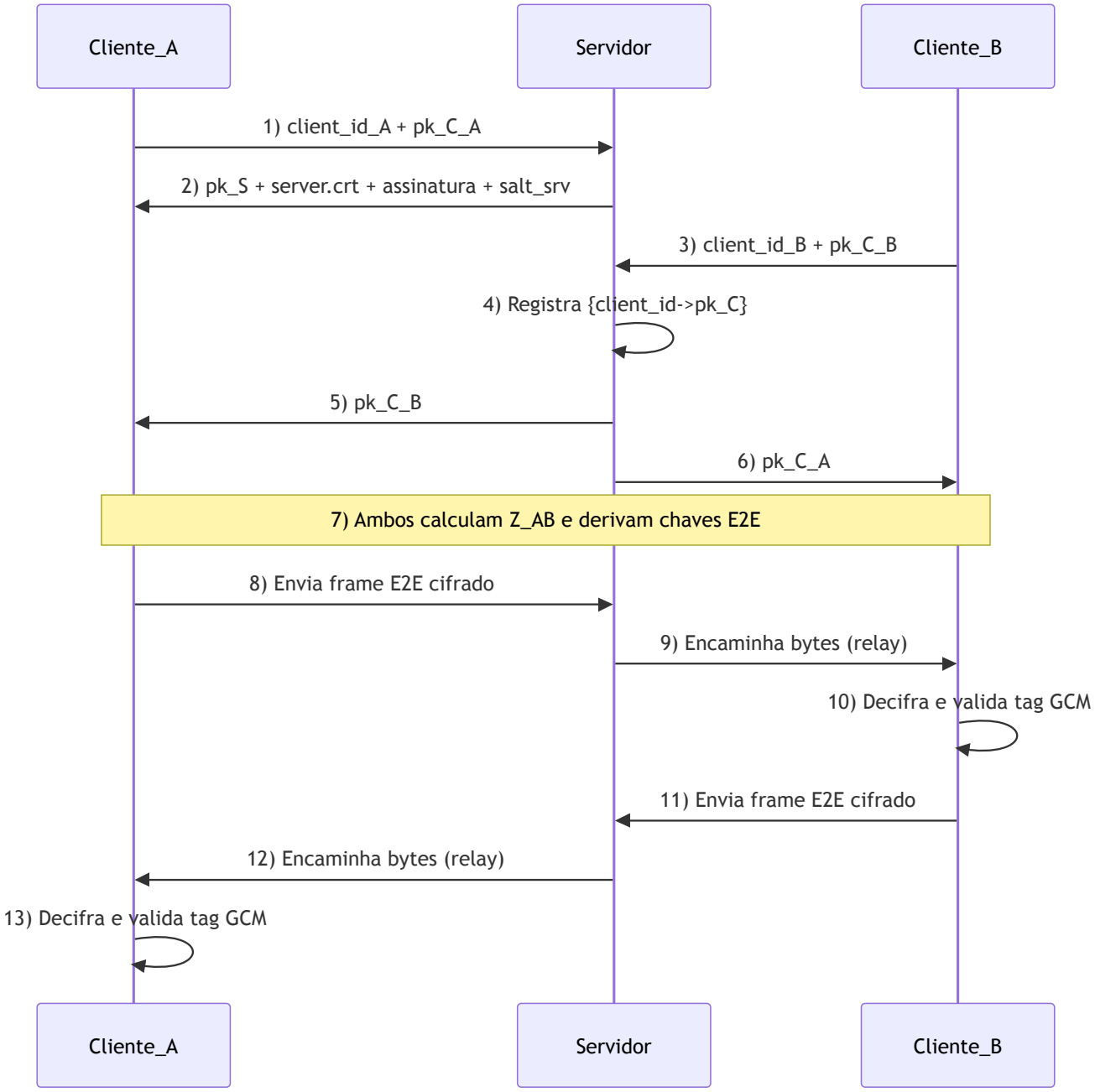
Cliente B:

```
peers = {
   client_id_A: {
      "public_key": pk_C_A (32B),
      "Z_AB": segredo ECDH com A (opcional armazenar),
      "key_send": Key_B2A (AES-128-GCM),
      "key_recv": Key_A2B (AES-128-GCM),
      "seq_send": próximo seq_no para enviar mensagens a A,
      "seq_recv": último seq_no recebido de A,
      "IV_base_send": IV_base para direção B→A,
      "IV_base_recv": IV_base para direção A→B
   }
}
```

Explicação dos campos:

- **public_key**: chave pública X25519 do peer.
- **Z_AB**: segredo ECDH (descartável depois de derivar chaves).
- **key_send / key_recv**: chaves simétricas derivadas via HKDF. `key_send` para enviar para A e `key_recv` para receber de A.
- **seq_send**: contador monotônico para mensagens enviadas.
- **seq_recv**: ultimo seq_no aceito (anti-replay).
- **IV_base_send / IV_base_recv**: IV base aleatório por direção.

Fluxo do Protocolo (Visão Geral)



Fluxo do Protocolo (Detalhado)

1. Inicialização

- Cliente abre conexão TCP com o servidor.
- Servidor aguarda múltiplas conexões simultâneas.

2. Handshake e Distribuição de Chaves

- **Cliente envia:** `client_id + pk_C` .
- **Servidor responde:** `pk_S + server.crt` + assinatura RSA + `salt_srv` .
  - Assinatura RSA-PSS com SHA-256 sobre o hash SHA256(pk_S || pk_C || client_id || salt)
  - Função da assinatura:
    - Provar autenticidade do servidor
    - Impedir MITM no registro das chaves públicas
- **Cliente valida**: certificado pinado + assinatura RSA-PSS
- **Servidor registra**: `client_id + pk_C` na tabela `sessions` .
  - Inicialização dos contadores na Tabela `sessions` no servidor:
    - `seq_send` = 0
    - `seq_recv` = -1
- **Servidor envia** `pk_C_B` para Cliente A e `pk_C_A` para Cliente B.

OBSERVAÇÃO: Sempre que um cliente novo se conecta ao servidor:

- O **servidor registra** `client_id + pk_C` na tabela `sessions` ,
- O **servidor envia** a chave pública do novo cliente para os outros clientes e as chaves públicas dos outros clientes para o novo cliente.

3. Derivação das Chaves E2E

Todos os cliente derivam as chaves E2E para se comunicar com outros clientes e registram na tabela `peers` . As chaves E2E são derivadas via HKDF.

- `Z_AB` = ECDH(sk_A, pk_B)
- `PRK` = HKDF-Extract(salt_srv, Z_AB)
- `Key_A2B` , `Key_B2A` derivadas via HKDF

Inicializaçã dos contadores e IVs na Tabela `peers` em cada par de clientes:

- `seq_send` = 0
- `seq_recv` = -1
- `IV_base_send` = random(4B)
- `IV_base_recv` = random(4B)

4. Envio de Mensagens

Cliente A -> Cliente B

- Cliente A incrementa `seq_send`, constrói `nonce` e cifra com `Key_A2B`.
- Servidor encaminha sem decifrar.
- Cliente B valida `seq_no`, reconstrói `nonce` e decifra com `Key_A2B`.

Cliente B -> Cliente A

- Cliente B incrementa `seq_send`, constrói `nonce` e cifra com `Key_B2A`.
- Servidor encaminha sem decifrar.
- Cliente A valida `seq_no`, reconstrói `nonce` e decifra com `Key_B2A`.

5. Encerramento

- Cliente desconecta → servidor remove entrada da tabela `sessions`.
- Nonces e seq_no não podem ser reutilizados.
- Opcional: rotação de chaves após N mensagens ou T minutos.

6. Garantias

- **Confidencialidade:** AES-GCM.
- **Integridade:** tag GCM.
- **Autenticidade:** certificado RSA.
- **Sigilo perfeito:** ECDHE.
- **Anti-replay:** seq_no monotônico.
- Servidor não consegue decifrar mensagens dos clientes.

Checklist para a apresentação

- Apresentação dos integrantes (cada aluno fala e apresenta parte do trabalho).
- Explicação do protocolo (handshake ECDHE + RSA + HKDF e troca de mensagens AES-GCM).
- Demonstração prática: inicialização do servidor, conexão de múltiplos clientes, envio de mensagens.
- Validação de segurança: mostrar que apenas o destinatário consegue decifrar.
- Estrutura de dados do servidor: explicar como sessões, chaves e contadores são gerenciados.

- Explicação do código: cada aluno apresenta trechos diferentes (handshake, derivação de chaves, criptografia, roteamento).
- Encerramento: resumo das garantias de segurança alcançadas.

Especificações operacionais e critérios de avaliação

1. Testes mínimos obrigatórios (apresentação)
    - Conectar pelo menos 2 clientes e trocar mensagens E2E: sucesso se destinatário decifra corretamente.
    - Mensagem adulterada (bitflip) no tráfego: destinatário deve rejeitar (GCM tag fail).
    - Replay: reenvio de frame com seq_no já usado deve ser rejeitado pelo destinatário (ou pelo servidor se optar por checar seq_no como parte do protocolo de roteamento).
    - Framing inválido (tamanho incorreto) deve ser tratado como erro e desconexão segura.
2. Política de logs
    - Não registrar chaves privadas, segredos compartilhados (Z), salt em texto claro, nonces ou conteúdo de mensagens.
    - Logs permitidos: eventos de conexão/desconexão, client_id, tamanhos de mensagem, razões para rejeição (ex: tag mismatch), e hashes/fingerprints de chaves para debug.
3. Limites e parâmetros
    - Salt length: 16 bytes.
    - Nonce length: 12 bytes (construção conforme especificada acima).
    - Max plaintext por mensagem: 4096 bytes (alunos podem justificar outro limite, mas documentar).
    - Número máximo de clientes concorrentes: livre, mas indicar que implementação deve tratar erros ao atingir recursos.
4. Reconexão e reuso de client_id
    - Ao desconectar, a sessão em memória é limpa. Se um cliente reconectar com o mesmo `client_id`, deve realizar handshake/registro completo e novas chaves E2E devem ser derivadas (não reutilizar salt ou seq_no anterior).
5. Grading rubric:
    - Implementação correta do handshake e rendezvous E2E: 30%
    - Implementação correta de AES-GCM E2E com nonce e seq_no: 25%
    - Proteções contra replay e tampering demonstradas: 20%
    - Código bem organizado, documentação e demo: 15%
    - Práticas de segurança (logs, não expor chaves): 10%